

Large-Scale JSON Tree Retrieval: A Single-Host Storage Microbenchmark

Junyao Dong

Abstract. Large JSON tree stores need fast node lookup, child expansion, subtree retrieval, and entity fetch. We benchmark their storage-level equivalents on one ten-million-node synthetic tree with warm caches and a local client. MongoDB’s short reads have higher fixed request and query setup than PostgreSQL, while subtree retrieval also pays more per-key cursor work and covered-row handoff. Against sequential individual queries, coalescing three already-known inputs reduces MongoDB group-completion P50 by $2.66\times$, $2.27\times$, and $2.62\times$ for node, child, and entity reads; at 64 inputs the corresponding ratios are $24.1\times$, $6.81\times$, and $25.1\times$. With at most 16 client workers, a threaded control still leaves coalescing 1.56 – $2.28\times$ faster at three inputs across two grouping seeds. Cursor batch size has no projection-independent optimum: a one-million-row request gives a modest ID-only improvement but not a covered-Metadata improvement. A frozen 256-row DFS bucket layout improves large-subtree P50/P95 by $1.57\times/1.72\times$ on the full root-ID-disjoint split; after removing all ancestor-correlated roots, the ratios remain $1.56\times/1.67\times$. Small subtrees regress. The practical direction is to coalesce naturally grouped short reads and route only large subtrees through buckets. The SQLite-backed Beam path now coalesces its per-turn child and entity reads. With the public defaults (`beam_size=3`, `select_k=1`), a file-backed control returns one leaf and its content, reduces batchable reads from 37 to five and all SQLite SELECTs from 39 to seven, and improves P50 by 1.177 – $1.192\times$. At 16 clients, median throughput improves from 237.3 to 272.2 queries/s, although absolute throughput falls as client count rises. Additional trees and mutable-tree support remain to be tested.

1 OPERATIONS AND LAYOUTS

We benchmark fully materialized reads on a large tree. MongoDB and PostgreSQL are compared under Materialized Path; PostgreSQL and SQLite are also measured under Order/Size. Inputs, ordering, returned fields, and logical results are identical. Entity fetch is layout-independent because the two hierarchy layouts in each SQL engine call the same shared Text query.

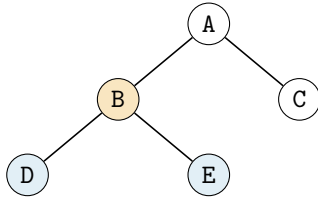
Table 1: The four storage operations measured in every configuration.

Operation	Returned data	Retrieval role
<code>get_node</code>	One node’s Structure and Metadata	Read or navigate to one node
<code>get_children</code>	Ordered direct-child Metadata	Choose the next branch
<code>get_subtree</code>	Ordered descendant Metadata	Render a tree view or retrieval block
<code>get_entity</code>	One selected node’s Text	Read final content

We compare two established hierarchy representations. Materialized Path stores each node’s root-to-node path as a string.¹ Order/Size labeling assigns each node a preorder rank and subtree size.² Our implementation stores the equivalent exclusive bound `end = pre + size`; a node v is a descendant of u when $\text{pre}(u) < \text{pre}(v) < \text{end}(u)$. Both representations keep Structure and Metadata together. MongoDB and PostgreSQL use dedicated Text stores; the two SQLite layouts share the original source table for entity fetch.

¹MongoDB, “Model Tree Structures with Materialized Paths.”

²Q. Li and B. Moon, “Indexing and Querying XML Data for Regular Path Expressions,” VLDB 2001.



(a) Logical tree

node	path
A	/A
B	/A/B
D	/A/B/D
E	/A/B/E
C	/A/C
/A/B/ <= path < /A/B0	

(b) Materialized path

node	pre	end
A	1	6
B	2	5
D	3	4
E	4	5
C	5	6
2 < pre < 5		

(c) Order/Size labeling

Figure 1: The same logical tree and its two hierarchy encodings for `get_subtree(B)`. Orange marks the query root; blue marks the returned descendants. In both layouts the descendants occupy one contiguous index range. Common node fields and entity storage are omitted.

Table 2: Five measured engine/layout configurations.

Engine	Materialized Path	Order/Size
MongoDB	Nodes + Text collections; path string	Not tested
PostgreSQL	Nodes + Text tables; path string	Nodes + Text tables; pre , end
SQLite	Path nodes + shared source table	Order/Size nodes + shared source table

2 METHOD

2.1 Experimental setup

All runs use a dual-socket Intel Xeon Gold 6418H host with 48 physical cores, 96 hardware threads, and 1 TiB RAM. MongoDB 7.0.34 and PostgreSQL 16.14 run as localhost services; the base SQLite storage experiments use SQLite 3.50.4 in process. The production-path controls in Section 5.5 were run separately with system Python 3.10.12 and SQLite 3.37.2; their raw files record the exact environment. Database and operating-system caches are not flushed.

The base layout and root-cause timings are sequential and single-client. The short-read section adds one 16-worker-capped group-completion control, and Section 5.5 separately measures throughput at one, four, and 16 clients. Mixed reads/writes, longer-duration load, and configurations above 16 clients are not evaluated.

The benchmark uses point and parent indexes for node and child reads, Materialized Path or Order/Size indexes for subtree reads, and a point index for Text. The PostgreSQL and SQLite subtree indexes cover the returned Metadata. MongoDB’s subtree index also carries `node_id`, `title`, and `summary`, so its range scan does not require document fetch.

Each configuration is warmed up, measured in cyclically interleaved order for three repeats, and summarized from the per-input median. SQLite is an in-process reference and is not used for server-to-server ratios. Optimization experiments state their own repeat counts and also summarize per-input medians. Base layout comparisons do not report confidence intervals, so sub-millisecond differences there are descriptive. The cursor experiment reports descriptive root-bootstrap intervals for paired medians; they do not model hierarchy/branch clustering.

2.2 Dataset and workload

The benchmark uses the same synthetic ten-million-node tree in all five configurations. It has templated random content, fanout 6–14, and maximum depth eight. `get_node` and `get_entity` use the first 500 leaf IDs ordered by `node_id`; `get_children` and `get_subtree` use the same 200 deterministic roots. The point and entity results characterize these fixed inputs; text sizes are not stratified. The subtree arm exercises the expensive descendant-range core: it excludes the root, has no depth limit, returns `(node_id, title, summary)`, and averages 36,456.7 rows. This storage microbenchmark is not the public ConDB API, which includes the root and applies a caller-supplied depth limit. All four measured operations are storage-level experimental equivalents. The MongoDB layouts and DFS buckets are not implemented in the current SQLite-backed public API. Short-read coalescing for child and entity reads is implemented inside the Beam retrieval path, but the public navigation methods remain scalar. Method names, projected record shapes, and path/node ordering also differ from the public facade, which exposes `get_content` and slot-ordered children.

Each configuration contains one sealed logical tree; multi-tree execution is outside this experiment. Every result is fully materialized. All 21,000 timed results match in row count and SHA-256 fingerprint.

2.3 Root-cause controls

For `get_node` and `get_entity`, hit/miss pairs show how much work remains when no row is returned. PostgreSQL runs without statement preparation for the main comparison; prepared execution is included as a sensitivity check. The

entity test also varies Text from zero to 256 KiB. The benchmark’s default MongoDB query is left unhinted and uses IDHACK.

For `get_children`, a synthetic tree varies fanout from zero to 128. Covered ID, covered Metadata, and noncovered plans expose fixed setup, covered row processing, and the document-access path.

Each short read is then repeated as one hot query shape while CPU cycles are sampled from the running MongoDB binary. Its `gitVersion` matches the `GitOrigin-RevId` in the official `r7.0.34` source tag. The sampled functions identify execution paths; their inclusive call-graph shares are not additive. A command-boundary control separately measures the Python driver wrapper.

For `get_subtree`, a scalar count removes BSON row output and client materialization. Long-range CPU counters measure instructions and cycles per key. PostgreSQL `EXPLAIN` verifies the plan but is not used as a latency measurement.

3 LATENCY OF ALL FOUR OPERATIONS

MongoDB and PostgreSQL form the server-to-server comparison. SQLite runs in process and is shown only as an embedded layout reference; its absolute latency is not ranked against the two server engines.

Table 3: P50/P95 latency in milliseconds. Each cell is computed from per-input medians over three interleaved repeats. The MongoDB and PostgreSQL Path columns form the server-to-server comparison; the remaining SQL columns show within-engine layout effects. Lower is better.

	MongoDB	PostgreSQL		SQLite	
Operation	Path	Path	Order/Size	Path	Order/Size
get_node	0.196 / 0.220	0.092 / 0.107	0.082 / 0.093	0.012 / 0.016	0.008 / 0.010
get_children	0.252 / 0.292	0.100 / 0.113	0.099 / 0.117	0.046 / 0.062	0.067 / 0.092
get_subtree	18.1 / 208.1	13.6 / 164.9	12.5 / 156.8	9.94 / 124.3	9.84 / 124.9
get_entity	0.142 / 0.166	0.074 / 0.082	0.070 / 0.077	0.008 / 0.010	0.005 / 0.006

At P50, the MongoDB–PostgreSQL gaps are 0.104 ms for node lookup, 0.152 ms for child expansion, 4.5 ms for subtree retrieval, and 0.068 ms for entity fetch. The subtree gap reaches 43.2 ms at P95. The short-read gaps are small in absolute time; subtree retrieval is the only operation where scan and row-emission cost materially changes latency.

Hierarchy representation matters primarily for subtree retrieval. PostgreSQL Order/Size reduces subtree P50 from 13.6 to 12.5 ms; SQLite changes by only 0.10 ms. Node and child differences are at most 0.021 ms and are descriptive at this scale. The two entity columns in each SQL engine repeat one shared Text query, so their tiny differences are timing noise, not a hierarchy-layout effect. The integer-bound subtree result is consistent with lower key-processing cost, but the independently built layouts do not isolate that factor alone.

4 MONGODB EXCESS LATENCY

The controls in Table 4 distinguish fixed request setup from row-dependent scan and materialization work.

Table 4: Root-cause controls for the four operations. Child slopes compare MongoDB with unprepared PostgreSQL in μ s per returned child. The covered and noncovered child arms also differ in filter, projection, and index shape, so their contrast localizes an execution bundle rather than a pure fetch cost.

Operation	Quantitative control	What the control establishes
<code>get_node</code>	MongoDB hit minus miss: 12.5 μ s P50; command boundary: 176.5 of 249.8 μ s inside the command	Row fetch is a small increment; most observed wall time lies within the command path.
<code>get_children</code>	Covered intercept/slope: 263/1.64 vs. 117/0.97; noncovered: 264/2.16 vs. 109/0.99	MongoDB shows both a larger fixed component and a steeper returned-row component; the noncovered bundle is steeper again.
<code>get_subtree</code>	Scalar count: 7.44 vs. 4.34 ms; 597 vs. 259 cycles/key	The gap survives removal of BSON row output and tracks higher per-key scan work.
<code>get_entity</code>	0 vs. 1 KiB text: 0.228 vs. 0.224 ms in MongoDB; 0.096 vs. 0.097 ms in PostgreSQL	At the benchmark’s 1 KiB payload, text transfer is below run-scale variation; fixed lookup work dominates.

4.1 Node lookup

For `get_node`, the hit–miss increment is only 12.5 μ s, while 176.5 of 249.8 μ s lies inside the MongoDB command. Source-localization samples show `find` parsing, canonicalization, collection acquisition, `QueryPlanner::plan`, and stage-tree construction. Document fetch is a small part of this point lookup; fixed command and planning work dominates.

4.2 Child expansion

Both engines use a parent index followed by base-record lookup for `get_children`. MongoDB has a higher fitted intercept and a steeper covered-row slope; the noncovered arm is steeper again. Samples show `IndexScan`, `KeyString` decoding, projection, and response append for covered rows. The noncovered path additionally includes `FetchStage`, `WorkingSetCommon::fetch`, and record-store seek. Because the covered and noncovered arms also use different filters and indexes, their difference is an access-path cost rather than a pure fetch measurement.

4.3 Subtree retrieval

For `get_subtree`, MongoDB’s overhead has two components: per-key cursor and executor work during the range scan, and per-row handoff during covered result production.

Table 5: Subtree scan cost. Mean latency uses 200 ranges on the ten-million-node tree. CPU counters use a fixed 500,000-key long-string range.

Metric	MongoDB	PostgreSQL	Ratio
Mean scalar-count latency (ms)	7.44	4.34	1.71×
Instructions per key	2,438	1,121	2.17×
CPU cycles per key	597	259	2.30×

Samples place the work in WiredTiger cursor advance and key extraction, the storage adapter, and classic executor handoff. The largest hotspot is `__wt_btcur_next`. The scalar count and CPU counters show that MongoDB spends more work per scanned key, with covered-row handoff adding a second per-row cost.

4.4 Entity fetch

For `get_entity`, samples localize the fixed lookup cost to the `IDHACK` command path, including request parsing, canonicalization, collection acquisition, executor initialization, `IDHackStage`, and projection construction. The payload control shows no measurable P50 change at 1 KiB. At 256 KiB, latency rises to 0.391 ms in MongoDB and 0.619 ms in PostgreSQL. Fixed lookup work dominates only at the 1 KiB size used by the main workload.

5 OPTIMIZATION EXPERIMENTS

The profiles suggest two different interventions. Short reads can share fixed command setup by coalescing already-known inputs. Subtree reads may need a larger storage record rather than a different network transfer chunk. We test both ideas in the storage harness.

The primary short-read sweep disables PyMongo command monitoring and uses 80 random input groups, five cyclically interleaved repeats, and two grouping seeds. Its 19,200 timed baseline–candidate pairs pass exact output-equivalence checks; repeated groups and IDs are not independent samples. A separate monitored run records command counts and produces nearly the same wall-time curve. Thus the reported wall time does not include one listener callback per database command. Table 6 gives representative endpoints. A separate three-arm control at $B \in \{1, 3, 8, 64\}$ uses 40 groups, three repeats, two grouping seeds, and at most 16 workers to compare sequential individual queries, concurrently submitted individual queries, and one `$in` query; even its 64-input arm never exceeds 16 client threads.

Table 6: MongoDB intervention endpoints. Short-read latency is wall time for one group of 64 already-known inputs under a sequential individual-query baseline. Subtree rows use separate paired cohorts and projections. The last column is the ratio of baseline to candidate latency at the marginal P50 and P95; it is not a percentile of per-input speedups. Compare only within a row.

Operation / intervention	Baseline P50 / P95 (ms)	Candidate P50 / P95 (ms)	Ratio at P50 / P95
<code>get_node</code> , $B = 64$	15.104 / 15.448	0.628 / 0.652	24.06 / 23.68×
<code>get_children</code> , $B = 64$	19.279 / 19.972	2.831 / 2.949	6.81 / 6.77×
<code>get_subtree</code> , ID-only 1M cursor request	10.671 / 118.552	10.155 / 115.188	1.05 / 1.03×
<code>get_subtree</code> , 256-row bucket	23.011 / 248.597	14.665 / 144.292	1.57 / 1.72×
<code>get_entity</code> , $B = 64$	13.382 / 13.639	0.533 / 0.559	25.10 / 24.41×

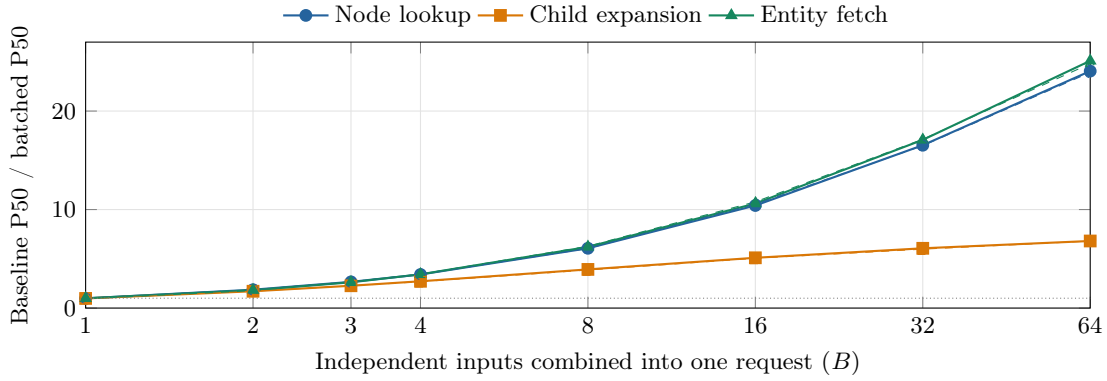


Figure 2: Wall-only batch-size sweep for the three short reads (80 groups, five repeats). The ratio compares B sequential equality queries with one output-equivalent `$in` query. Solid and dashed lines use two input grouping seeds in adjacent runs. The monotone trend, including the realistic $B = 3$ point, matches the expected effect of sharing fixed command setup. These are group-completion times, not concurrent-load throughput.

5.1 Node lookup

5.1.1 Optimization method. For each $B \in \{1, 2, 3, 4, 8, 16, 32, 64\}$, the baseline issues B equality `find` commands sequentially for a group of node IDs that is already known. The candidate sends the same IDs in one `$in` query, retains the `tree_id, node_id` index and projection, builds a map keyed by `node_id`, and restores caller input order. Thus index choice, returned records, and caller-visible order are held constant; the controlled change is B commands versus one multi-key command. The three-arm control submits those same B equality commands concurrently through the shared thread pool, preserving their query shape and output contract.

5.1.2 Experimental results. Node coalescing scales smoothly with group size. $B = 1$ is unchanged, $B = 3$ improves P50 from 0.729 to 0.274 ms (2.66 $\times$), and the ratio grows to 24.1 $\times$ at $B = 64$. The second grouping seed stays within 0.4% at $B = 3$ and 0.6% at $B = 64$. In the separate telemetry run, command count falls from B to one and the wall-time curve remains close to the listener-free result. The gain comes from sharing command and query setup, not from a faster point lookup. With the 16-worker cap, `$in` remains 2.05 $\times$ /1.79 $\times$ faster than concurrently submitted individual queries at $B = 3$ across the two seeds, and 17.9 $\times$ /18.3 $\times$ faster at $B = 64$.

This helps only when the caller already has a group of independent IDs. It does not speed up a single lookup, and the benchmark does not include time spent forming a group. Its end-to-end value will depend on real group sizes and API-level load tests.

5.2 Child expansion

5.2.1 Optimization method. The baseline performs one indexed, ordered scan sequentially for each parent ID. The candidate uses one `parent_id:{$in:...}` query, sorts by `parent_id, path, node_id`, groups rows by parent, and emits each parent’s children in the original path/node order. Both arms project the same child Metadata and use the same parent/path index. The candidate additionally pays for cross-parent ordering and client regrouping. Its threaded control submits the same per-parent scans concurrently through the shared pool without changing their individual query shape.

5.2.2 Experimental results. Returned-row work limits child coalescing. At $B = 3$, P50 improves 2.27 $\times$, but by $B = 64$ the ratio is 6.81 $\times$, far below the 24.1 $\times$ point-read result. An average 64-parent group still returns about 643 child documents, so removing 63 command setups leaves scanning, document production, cross-parent ordering, regrouping, and output materialization. The curve begins to flatten after $B = 16$ as those per-child costs take over. Against individual queries submitted through the 16-worker-capped pool, the two seeds retain 1.81 $\times$ /1.56 $\times$ at $B = 3$ and 4.96 $\times$ /5.07 $\times$ at $B = 64$. Bounded concurrency reduces neither the benefit of coalescing nor the returned-row ceiling.

Coalescing remains useful when several parent IDs are already available, but it will not reach point-read-scale gains. Further improvement must reduce per-child fetch or output work.

5.3 Subtree retrieval

5.3.1 Optimization methods. Cursor batching. The first intervention changes only `Cursor.batch_size` for the same descendant-range query, index, projection, sort, and complete materialization. Both ID-only and covered-Metadata arms use the same 100 row-count-stratified ranges and five cyclically interleaved repeats; every variant and repeat stores an ordered-output digest. Primary wall-time runs register no command listener. A separate telemetry run records reply counts and durations. Requested sizes are 101, 1K, 10K, and 1M, but actual replies remain bounded by MongoDB message limits. The two projections test whether any cursor result survives a larger returned-record shape rather than

treating ID-only behavior as universal. Root-bootstrap intervals resample ranges and are descriptive; they do not correct for shared hierarchy branches.

DFS buckets. The structural intervention packs consecutive DFS/path-order rows into parallel column arrays. A query performs the normal root lookup, two covered boundary probes, one contiguous bucket-sequence scan, and a range check on every decoded row; logically, only the two boundary buckets can contain out-of-range rows. Five candidate sizes (128, 256, 512, 2,048, and 8,192 rows) run together on 75 selection roots. The 256-row candidate has the lowest absolute selection P95; the winner is frozen before a separate runner measures 100 previously unqueried root IDs for seven paired repeats. Selection and holdout have no repeated root IDs, but they come from the same depth-1–3 large-subtree population on one tree and are not branch/range-disjoint: 19 selection–holdout ancestor relations affect 16 holdout roots. We report both the full frozen holdout and a conservative post-hoc sensitivity that removes every holdout root involved in any selection/holdout or within-holdout ancestor relation.

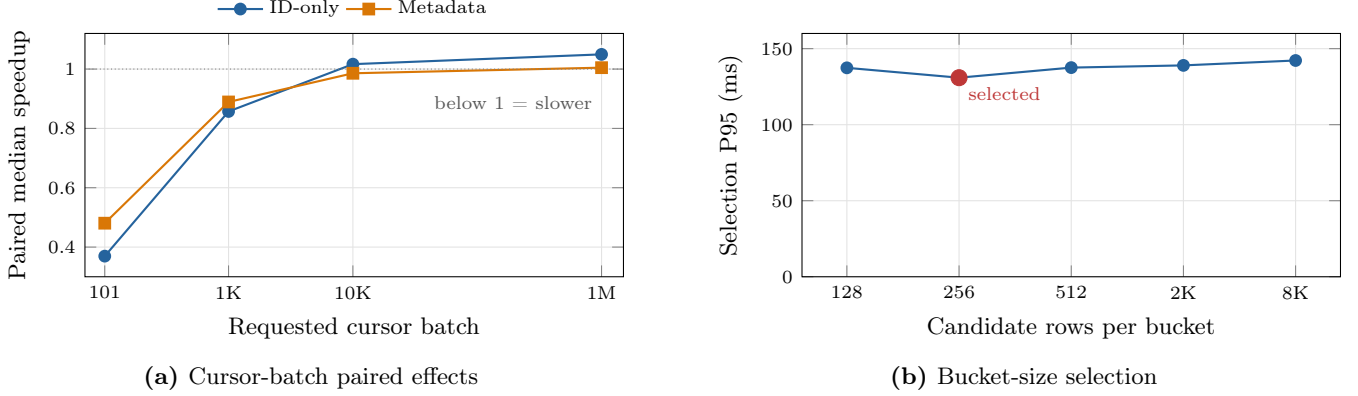


Figure 3: The two subtree parameter sweeps. (a) Points are medians of per-range paired speedups and vertical bars are 95% root-bootstrap intervals. Small cursor batches are consistently worse. The modest ID-only effect at 10K/1M does not transfer to covered Metadata. (b) The 256-row bucket has the lowest selection P95 in this run, but the five candidates span only 8.6%; selection alone does not establish a generally optimal size.

5.3.2 Experimental results. Small cursor batches hurt both projections. At 101 and 1K rows, paired median speedups are 0.369 and 0.857 for ID-only output and 0.481 and 0.889 for covered Metadata. Telemetry shows why: the driver default uses a median of 2 commands, including 1 `getMore`, while 1K uses 12 commands and 101 uses 117.

Larger requests help only the narrow ID-only case. Its paired median reaches 1.017 at 10K and 1.049 at 1M; the 1M interval is $[1.038, 1.056]$, with marginal P50/P95 ratios of $1.05\times/1.03\times$. Covered Metadata stays at 0.986 and 1.005, with intervals $[0.970, 1.001]$ and $[0.998, 1.010]$. We therefore keep the driver default for the covered subtree path. Remote RTT, streaming, or early termination may have a different tradeoff.

The bucket sweep changes the storage record itself. The selection curve is shallow: 256 rows has the lowest P95, but all five sizes lie within 8.6%. An earlier run selected 2,048 rows and produced nearly the same holdout P50/P95 ratios ($1.57\times/1.73\times$ versus $1.57\times/1.72\times$). The bucketed layout matters more than the exact bucket size in this range.

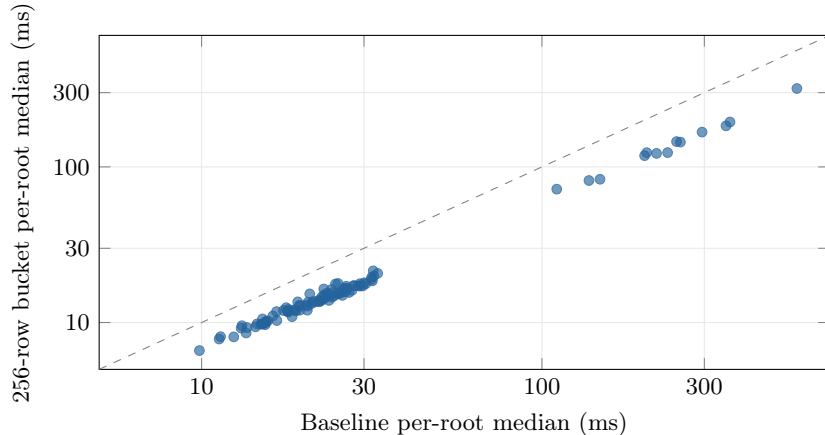


Figure 4: Paired DFS-bucket result for 100 previously unqueried, root-ID-disjoint inputs from the same large/shallow-subtree cohort. Coordinates are seven-repeat medians. The split is not branch/range-disjoint, so points are not 100 independent storage regions. All points fall below the equal-latency diagonal; logarithmic axes retain the cohort tail.

All 100 holdout roots improve. Marginal P50/P95 speedups are $1.57\times/1.72\times$, and 697 of 700 paired repeats are faster.

The split is root-ID-disjoint but not branch-disjoint. Removing every holdout root involved in a selection/holdout or within-holdout ancestor relation leaves 67 mutually range-disjoint roots. All 67 improve, with marginal P50/P95 speedups of $1.56\times/1.67\times$.

Fetch P95 falls from 172.5 to 71.2 ms because each BSON document carries many consecutive logical rows. The saving comes from fewer storage-engine record handoffs; boundary lookup, edge overfetch, array decoding, and client reconstruction remain.

Cost and validation. Rebuilding the 256-row layout produces 39,063 buckets in 77.0 s. A full pass rescans all 10M source rows, decodes the buckets, and reproduces the source digest with zero mismatches. The collection and its indexes occupy 1.450 GB, and the largest BSON document is 121.3 KiB. This adds 20.4% to the retained source collection and its five indexes; against the 1.672 GB source collection alone, the increase is 86.7%. A hybrid route retains the existing structures, so the former denominator matches that design.

Buckets are a poor fit for small subtrees. Across 250 previously unused roots returning 265 rows on average, P50 rises from 1.044 to 2.171 ms and only 30 roots improve, although P95 falls from 3.769 to 3.348 ms. A serving path should route by subtree size and keep small reads on the existing index. Mutable trees also need an update, rebuild, and atomic-publication design.

5.4 Entity fetch

5.4.1 Optimization method. For each batch size, the baseline issues sequential equality lookups on the Text collection's `_id`; the candidate sends one `_id:${in:...}` query. Both arms explicitly use the same `_id` index, return the same `(_id,text)` fields, and restore caller input order with an in-memory ID map. The threaded control concurrently submits the same individual `_id` lookups through the shared pool.

5.4.2 Experimental results. Entity fetch follows the same curve as node lookup. A one-item batch is unchanged, $B = 3$ improves P50 from 0.698 to 0.267 ms ($2.62\times$), and $B = 64$ reaches $25.1\times$; the second grouping seed stays within 2.0% at both points. Repeating the pattern in both the Nodes and Text collections points to shared command setup rather than a collection-specific effect. In the 16-worker-capped threaded control, the two seeds retain $2.28\times/2.12\times$ at $B = 3$ and $20.2\times/19.1\times$ at $B = 64$.

The main workload uses roughly 1 KiB texts. Larger entities encounter response-size and cursor-chunking costs, as the 256 KiB control in Section 4 shows. Multi-entity fetch is useful for naturally co-requested small texts.

5.5 Production-path integration

5.5.1 Optimization method. The SQLite storage layer now provides multi-parent-child and multi-entity reads. Inputs are deduplicated, missing entries remain explicit, every query is scoped by `tree_id`, and requests are split into 500-ID chunks. Child rows are regrouped by parent without changing slot order. Beam uses a batch method only when at least two uncached IDs are present; a one-ID read keeps the existing equality query. Nodes with a null `entity_id` do not issue an entity lookup. Storage implementations without the optional methods continue through the scalar path. Final contents bypass the cross-call entity cache, so batching does not weaken the previous fresh-read behavior.

The same change fixes a result-selection bug exposed by the first control. Previously, a ranked internal node was inserted into the final result even when the ranker returned `done=false`; with the default `select_k=1`, retrieval stopped before the next frontier. An internal node now advances to the next turn when `done=false`. A leaf or file is a final result, while an internal document node is final only when the ranker returns `done=true`, meaning that its content is already specific enough.

The latency control uses file-backed `TreeDB` copies and the real `BeamRetriever`. It keeps the public defaults `beam_size=3`, `select_k=1`, and the tree-depth turn limit. The tree has eight first-level sections and eight leaves per section; every section and leaf has a 1 KiB entity. A fixed ranker retains three sections on turn one with `done=false`, then selects one leaf on turn two. Both arms return exactly one leaf and one content object. The scalar arm hides the two batch methods; the candidate arm exposes them. SQL tracing is used only for an untimed count. Five rounds contain 250 paired repetitions per arm after 30 warmups, with arm order alternating within each round.

Table 7: File-backed Beam control with the public `beam_size` and `select_k` defaults. Times cover the complete deterministic two-turn retrieval. The ratio is scalar latency divided by batched latency.

Round	Scalar P50 / P95 (ms)	Batched P50 / P95 (ms)	Ratio P50 / P95
1	0.735 / 0.865	0.618 / 0.728	1.189 / 1.188×
2	0.732 / 0.861	0.615 / 0.726	1.191 / 1.186×
3	0.735 / 0.859	0.625 / 0.727	1.177 / 1.181×
4	0.726 / 0.807	0.610 / 0.697	1.191 / 1.157×
5	0.725 / 0.857	0.608 / 0.718	1.192 / 1.194×

5.5.2 Experimental results. The scalar arm performs one root-child read, eight first-turn entity reads, three frontier-child reads, 24 second-turn entity reads, and one final entity read. These 37 batchable reads become five. The shared root-ID and maximum-depth lookups bring the full counts to 39 and seven `SELECT`s. P50 improves by 1.177–1.192× across the five rounds, a 15.1–16.1% latency reduction; P95 improves by 1.157–1.194×, a 13.5–16.3% reduction. The smaller wall-time gain is expected: prompt rendering, candidate construction, JSON decoding, and result materialization are unchanged.

A second control opens one SQLite connection per client and runs 100 queries per client at one, four, and 16 clients. Three rounds alternate arm order. The 16-client limit is enforced before a database is opened or a worker is created. Table 8 reports the median arm result across the three rounds.

Table 8: File-backed load control. Throughput and P50 latency are medians across three rounds. Ratios are the medians of the three within-round paired ratios: batched/scalar for throughput and scalar/batched for latency.

Clients	Scalar / batched q/s	Throughput ratio	Scalar / batched P50 (ms)	Latency ratio
1	1224.7 / 1501.4	1.207×	0.788 / 0.629	1.213×
4	443.3 / 551.3	1.239×	9.108 / 7.234	1.257×
16	237.3 / 272.2	1.157×	67.168 / 58.590	1.156×

Every paired round favors batching. At 16 clients, the medians of the paired throughput and P50 ratios are 1.157× and 1.156×. The separately aggregated arm medians correspond to 14.7% higher throughput and 12.8% lower P50 latency. Median arm P95 falls from 75.128 to 65.743 ms; the median paired P95 ratio is 1.147×. Neither arm scales with client count: contention and Python-side work drive throughput down and latency up. Coalescing reduces that cost but does not make the current SQLite path a high-concurrency design.

The raw results are stored in:

```
bench/db/sqlite_beam_batching_20260729.json
bench/db/sqlite_beam_batching_load_20260729.json
```

Each records its command line, Git head, output fingerprint, and source-file hashes. The public scalar navigation methods, other tree shapes, and mutable trees remain outside this control.

6 CONCLUSION

Request coalescing is now integrated into the SQLite-backed Beam path. The storage experiment shows that it helps with only three inputs and remains faster than submitting individual queries through a 16-worker pool. The production control confirms the direction but also bounds the claim: reducing 37 batchable reads to five yields a repeatable 1.18× single-client gain and a 1.16× median paired throughput gain at 16 clients, not a statement-count-sized speedup. Cursor batch tuning is not worth pursuing for the covered subtree path: small batches add `getMore` traffic, while large requests fail to improve Metadata retrieval.

DFS buckets are useful for large subtrees but should sit behind a size-based route because small reads regress. The next experiments should measure the same Beam path on additional trees, decide whether public navigation needs batch methods, and evaluate bucket routing on branch-disjoint trees with mixed subtree sizes. Mutable trees also need measured rebuild and publication behavior.